INTRODUCTION TO JAVA

From Zero to Hero

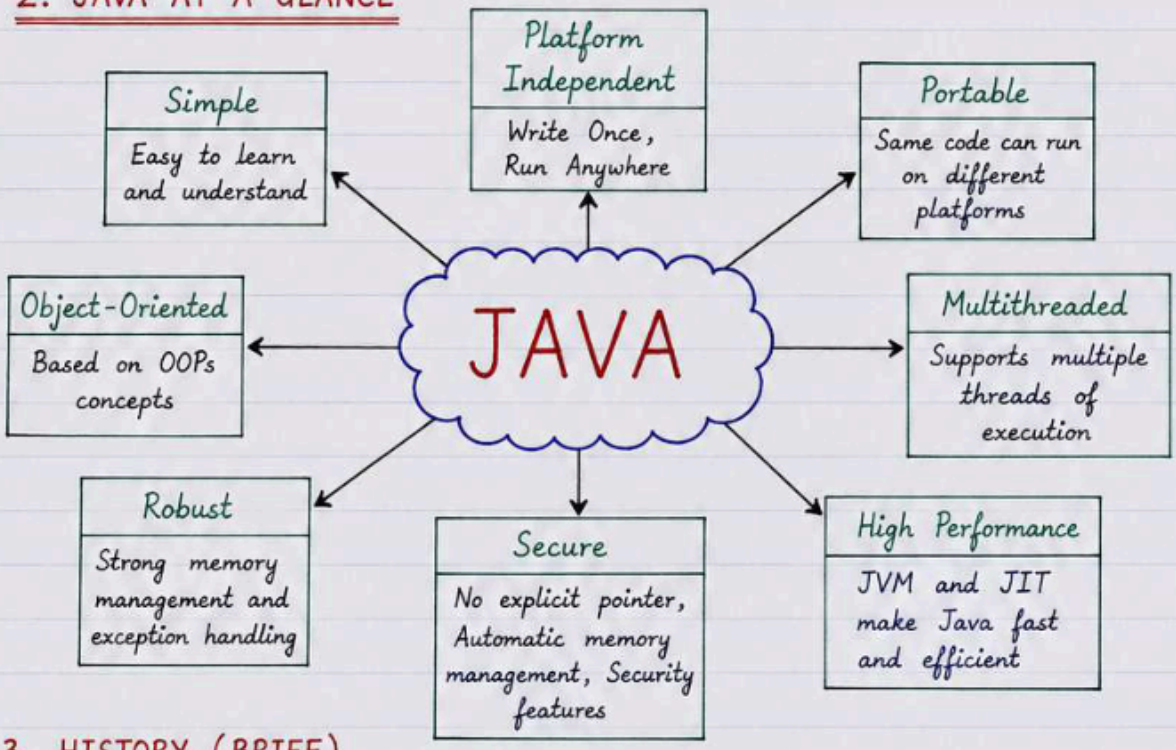
" Java is a high-level, class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. "

IMP

1. WHAT IS JAVA ?

→ Java is a high-level programming language.
→ It is simple, secure, portable and platform independent.
→ Java follows the principle of OOPs (Object Oriented Programming).
→ Java is widely used for building web, mobile, desktop, enterprise and various standalone applications.

2. JAVA AT A GLANCE



3. HISTORY (BRIEF)

→ Java was developed by James Gosling at Sun Microsystems in 1995.
→ Originally it was called "Oak".
→ Later the name was changed to "Java".
→ In 2009, Oracle acquired Sun Microsystems.
→ Java is one of the most popular programming languages in the world.

4. WHERE IS JAVA USED ?

- Web Applications
- Mobile Applications (Android)
- Desktop Applications
- Enterprise Applications
- Scientific Applications
- Banking & Finance Systems
- Big Data Technologies
- Cloud Applications

INTRODUCTION TO JAVA

From Zero to Hero

2. JVM, JRE & JDK

IMP

"To understand Java deeply, first understand how Java runs."

1. JVM (JAVA VIRTUAL MACHINE)

→ JVM is the heart of Java.
→ It is a virtual machine that runs Java bytecode.
→ It provides platform independence.
→ It performs class loading, bytecode verification, code execution, memory management, etc.

2. JRE (JAVA RUNTIME ENVIRONMENT)

→ JRE is the implementation of JVM.
→ It includes everything required to run a Java program.
→ It contains JVM + core libraries + other supporting files.

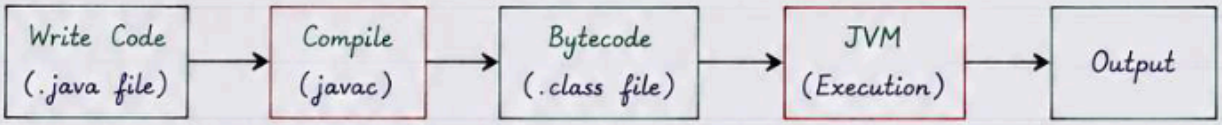
3. JDK (JAVA DEVELOPMENT KIT)

→ JDK is a complete package.
→ It is used by developers to develop, compile, debug and run Java programs.
→ It contains JRE + development tools.

In Simple Words

- JVM – Runs the Java code
- JRE – Runs the Java program
- JDK – Develops + Runs the Java program

4. HOW JAVA PROGRAM EXECUTES ?



5. EXECUTION FLOW OF A JAVA PROGRAM

→ Developer writes code in .java file.
→ Compiler (javac) converts it into bytecode (.class file).
→ JVM loads .class file and executes the code.
→ JVM converts bytecode into machine specific code (if required).
→ Output is displayed.

Key Points

- Java code is compiled, not interpreted.
- JVM makes Java platform independent.
- Bytecode is executed by JVM.

6. JVM vs JRE vs JDK (COMPARISON)

Feature	JVM	JRE	JDK
Full Form	Java Virtual Machine	Java Runtime Environment	Java Development Kit
Purpose	Runs Java bytecode	Runs Java programs	Develops, compiles, runs and debugs programs
Contains	Class Loader, Bytecode Verifier, Execution Engine, Runtime Data Areas	JVM + Core Libraries + Other Files	JRE + Development Tools (Compiler, Debugger, etc.)
Used By	Internally by Java (End Users)	End Users	Developers
Size	Smallest	Medium	Largest
Example (Windows)	java.exe	javaw.exe, rt.jar, etc.	javac.exe, java.exe, tools, etc.

INTRODUCTION TO JAVA

From Zero to Hero

3. JAVA PROGRAM STRUCTURE & FIRST PROGRAM

"Every Java program begins with a class and the execution starts from the main() method."

1. BASIC STRUCTURE OF A JAVA PROGRAM

```
class HelloWorld ──────────────────────→ Class Declaration
{
  public static void main(String[] args) ──→ main() method
  {                                           (Entry Point)
    System.out.println("Hello World!"); ──→ Statement / Code
  }                                    ──→ End of main()
}                                      ──→ End of class
```

2. EXPLANATION

- class – Blueprint / template for creating objects.
- main() – Entry point of every Java program.
- public – Accessible everywhere.
- static – Can be called without creating object.
- void – Does not return any value.
- System.out.println() – Used to print output on screen.

3. COMPONENTS EXPLAINED

★ class – A collection of variables and methods.

★ main() – JVM looks for this method to start execution.

★ String[] args – Command line arguments (optional).

★ System.out.println() – Prints text to the console.

4. COMMENTS IN JAVA

Type	Syntax	Example	Use
Single Line Comment	// comment	// This is single line comment	For short comment
Multi Line Comment	/* comment */	/* This is multi line comment */	For multiple line comment
Documentation Comment	/** comment */	/** This is doc comment */	For generating documentation

5. IDENTIFIERS & NAMING RULES

- An identifier is a name used for class, variables, methods, etc.
- Can contain letters (A-Z, a-z), digits (0-9), underscore (_), dollar ($).
- Must start with letter, _ or $ (not with digit).
- No special characters allowed.
- Case sensitive.

Example:
validName, _count, $price1, totalMarks

Invalid:
1name, total-marks, %value

6. HOW A JAVA PROGRAM IS COMPILED & EXECUTED

1. Write Code — We write code in .java file using any editor.
→ 2. Compile Code — javac compiler converts .java file to .class bytecode file.
→ 3. Bytecode — .class file contains bytecode which is platform independent.
→ 4. Execute — JVM executes bytecode and produces output.
→ 5. Output — Result is displayed on the console.

Key Points

- .javac is the compiler.
- .java is the runner (JVM).
- .java → .class → Execution → Output
- Bytecode runs on any platform.
- This is why Java is platform independent (Write Once, Run Anywhere).

4. VARIABLES & DATA TYPES

1. WHAT IS A VARIABLE ?

→ A variable is a named memory location used to store data.

→ Its value can change during program execution.

→ Every variable has a data type which defines the type of data it can store.

2. DECLARATION vs INITIALIZATION

Declaration	Initialization
Tells the compiler about the variable name and type. Syntax : dataType variableName;	Assigns an initial value to the variable. Syntax : variableName = value;

3. RULES FOR NAMING VARIABLES

- Can contain letters, digits, _ and $.
- Must start with a letter, _ or $.
- No spaces allowed.
- Case sensitive (age, Age and AGE are different).
- Should not be a keyword.

4. DATA TYPES IN JAVA

→ Java has two types of data types.

1. Primitive Data Types (built-in)
2. Reference Data Types (non-primitive)

5. PRIMITIVE DATA TYPES

Type	Size	Default Value	Range	Description
byte	1 byte	0	-128 to 127	Stores whole numbers
short	2 bytes	0	-32,768 to 32,767	Stores whole numbers
int	4 bytes	0	-2^{31} to $2^{31}-1$ (-2147483648 to 2147483647)	Most commonly used integer type
long	8 bytes	0L	-2^{63} to $2^{63}-1$ (very large numbers)	Used for very large integer values
float	4 bytes	0.0f	~1.4E-45 to 3.4E+38	Stores decimal numbers
double	8 bytes	0.0d	~4.9E-324 to 1.7E+308	More precision than float
char	2 bytes	'\u0000'	0 to 65535	Stores a single character
boolean	1 bit	false	true or false	Stores true or false

6. REFERENCE DATA TYPES

→ These are Non-primitive types.

→ They do not store actual values, but reference (address) of objects.

→ Examples : String, Arrays, Classes, Interfaces, etc.

7. DEFAULT VALUES OF VARIABLES

For Numeric Types (byte, short, int, long)	→ 0
For float and double	→ 0.0
For char	→ '\u0000'
For boolean	→ false
For Reference types	→ null

8. TYPE CASTING

→ WIDENING (IMPLICIT)

- Automatically done by compiler.
- From smaller type to larger type.
- No data loss.

Example : int → long , float → double

→ NARROWING (EXPLICIT)

- Manually done by the programmer.
- From larger type to smaller type.
- May cause data loss.

Example : double → int , long → short

9. LITERALS

→ Fixed values used in a program are called literals.

Examples : 10 , 20.5 , 'A' , true , "Hello"

QUICK SUMMARY

- Variables store data.
- Every variable has a type.
- Primitive types store actual values.
- Reference types store addresses.
- Type casting helps in type conversion.

5. OPERATORS IN JAVA

"Operators are special symbols that perform operations on variables and values."

1. ARITHMETIC OPERATORS

Operator	Name	Example	Meaning
+	Addition	a + b	Sum
-	Subtraction	a - b	Difference
*	Multiplication	a * b	Product
/	Division	a / b	Quotient
%	Modulus	a % b	Remainder
++	Increment	a++	Increase by 1
--	Decrement	a--	Decrease by 1

- '/' gives quotient, '%' gives remainder.

2. RELATIONAL OPERATORS

Operator	Name	Example	Meaning
==	Equal to	a == b	True if a is equal to b
!=	Not equal to	a != b	True if a is not equal to b
>	Greater than	a > b	True if a is greater than b
<	Less than	a < b	True if a is less than b
>=	Greater than or equal to	a >= b	True if a is greater than or equal to b
<=	Less than or equal to	a <= b	True if a is less than or equal to b

- Relational operators always return boolean value (true / false).

3. LOGICAL OPERATORS

Operator	Name	Example	Meaning
&&	Logical AND	(a > 5 && b < 10)	True if both conditions are true
||	Logical OR	(a > 5 || b < 10)	True if any one condition is true
!	Logical NOT	!(a > b)	Reverses the boolean value

- Used to combine multiple conditions.

4. ASSIGNMENT OPERATORS

Operator	Example	Equivalent To
=	a = b	a = b
+=	a += b	a = a + b
-=	a -= b	a = a - b
*=	a *= b	a = a * b
/=	a /= b	a = a / b
%=	a %= b	a = a % b

- These operators save time and make code clean.

5. UNARY OPERATORS

- ++ (Increment)
  - Pre-increment : ++a
  - Post-increment : a++
- -- (Decrement)
  - Pre-decrement : --a
  - Post-decrement : a--
- + (Unary Plus)
  - Indicates positive value.
- - (Unary Minus)
  - Changes the sign.
- ! (Logical NOT)
  - Reverses boolean value.

6. TERNARY OPERATOR

→ Also called conditional operator.

→ Syntax :

```
condition ? expression1 : expression2
    ↓           ↓           ↓
 If true   execute this   If false
```

→ Example :

```
int max = (a > b) ? a : b;
```

(If a > b true then a will be assigned to max otherwise b)

7. BITWISE OPERATORS (Work on bits)

Operator	Name	Example (a = 5, b = 3)	Meaning
&	Bitwise AND	a & b = 5 & 3 = 1	Compares each bit. 1 if both bits are 1.
|	Bitwise OR	a | b = 5 | 3 = 7	1 if any one of the bits is 1.
^	Bitwise XOR	a ^ b = 5 ^ 3 = 6	1 if bits are different.
~	Bitwise Complement	~a = ~5 = -6	Inverts all the bits.
<<	Left Shift	a << 1 = 5 << 1 = 10	Shifts bits left by specified positions.
>>	Right Shift	a >> 1 = 5 >> 1 = 2	Shifts bits right by specified positions.
>>>	Unsigned Right Shift	5 >>> 1 = 2	Fills 0 on left side.

8. OPERATOR PRECEDENCE (Higher precedence executes first)

( ) > ++ -- > * / % > + - > < <= > >= > == != > && > || > =

(Highest) (Lowest)

- Use parentheses to avoid confusion and improve readability.

6. CONDITIONAL STATEMENTS

"Conditional statements are used to make decisions in a program."

1. if STATEMENT

→ Executes a block of code if the condition is true.

→ If condition is false, the block is skipped.

```
if (condition) {
    // code to be executed
}
```

2. if - else STATEMENT

→ Executes one block if condition is true otherwise executes another block.

```
if (condition) {
    // code if true
} else {
    // code if false
}
```

3. else - if LADDER

→ Used to check multiple conditions.

→ Executes the first true condition block.

```
if (condition1) {
    // code
} else if (condition2) {
    // code
} else if (condition3) {
    // code
} else {
    // code
}
```

4. NESTED if STATEMENT

→ An if or else-if inside another if or else.

→ Used for complex decision making.

```
if (condition1) {
    if (condition2) {
        // code
    } else {
        // code
    }
} else {
    // code
}
```

5. switch STATEMENT

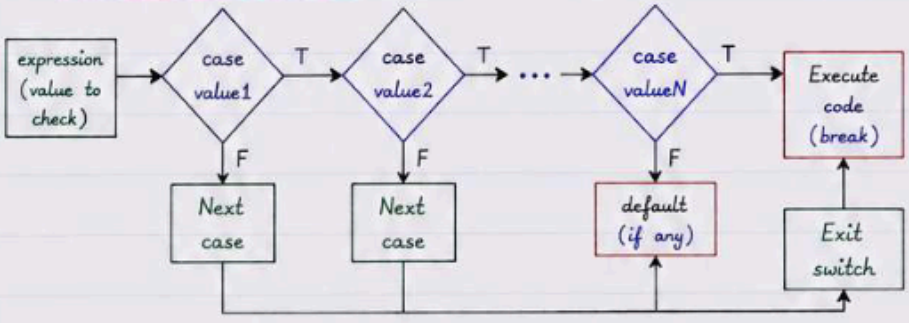
→ Used when we have multiple choices based on a single value.

→ Improves readability compared to else-if ladder.

→ Works with byte, short, int, char, enum, String (from Java 7).

```
switch (expression) {
    case value1 :
        // code
        break ;
    case value2 :
        // code
        break ;
    ....
    default :
        // code
}
```

How switch works ?



6. break, continue & default

Keyword	Purpose	Used In
break	Exits from loop or switch.	loop, switch
continue	Skips current iteration and moves to next.	loops
default	Executes when no case matches.	switch

When to use what ?

- Use if when condition is simple.
- Use if-else when there are two paths.
- Use else-if ladder when multiple conditions.
- Use switch when checking equality against many values.

→ if, if-else, else-if are flexible and can work with any condition.

→ switch is faster and cleaner when comparing a single variable with multiple values.

7. LOOPS IN JAVA

"Loops are used to execute a block of code repeatedly until a certain condition is met."

1. WHY LOOPS ?

→ To repeat a block of code multiple times.
→ To avoid writing same code again and again.
→ Saves time and reduces code length.
→ Essential for solving real world problems.

2. TYPES OF LOOPS IN JAVA

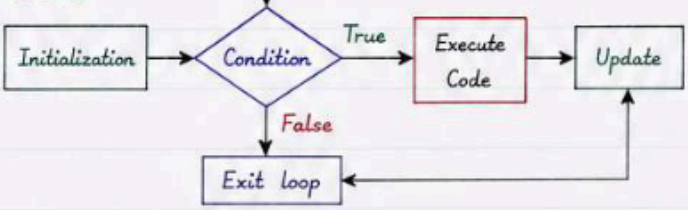
- for loop
- while loop
- do-while loop
- Enhanced for loop (for-each loop)

3. FOR LOOP

```
for (initialization ; condition ; update) {
    // code to be executed
}
```

→ initialization → executed once at the beginning.
→ condition → checked before each iteration.
→ update → executed after each iteration.

Flow :



Example : Print numbers from 1 to 5

```
for (int i = 1 ; i <= 5 ; i++) {
    System.out.println(i);
}
```

Output : 1 2 3 4 5

4. WHILE LOOP

```
while (condition) {
    // code to be executed
    // update
}
```

→ Entry controlled loop.
→ Condition is checked first.
→ If condition is false initially, code will not execute even once.
→ Update part is written manually inside the loop.

Example : Print numbers from 1 to 5

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;   // update
}
```

Output : 1 2 3 4 5

5. DO-WHILE LOOP

```
do {
    // code to be executed
    // update
} while (condition);
```

→ Exit controlled loop.
→ Condition is checked after executing the block.
→ Code executes at least once even if condition is false.
→ Semicolon (;) is mandatory at the end.

Example : Print numbers from 1 to 5

```
int i = 1;
do {
    System.out.println(i);
    i++;   // update
} while (i <= 5);
```

Output : 1 2 3 4 5

6. COMPARISON OF LOOPS

Feature	for loop	while loop	do-while loop
Type	Entry controlled	Entry controlled	Exit controlled
Condition check	Before execution	Before execution	After execution
Minimum execution	Zero or more times	Zero or more times	At least once
Best use	When number of iterations is known	When number of iterations is not known	When code must run at least once
Update	In update section	Inside the loop	Inside the loop

7. BREAK AND CONTINUE IN LOOPS

★ break – Terminates the loop immediately.
→ Control moves to the statement after the loop.

```
for (int i = 1 ; i <= 10 ; i++) {
    if (i == 5) break;
    System.out.println(i);
}
```

Output :
1 2 3 4

★ continue – Skips the current iteration.
→ Control moves to the next iteration.

```
for (int i = 1 ; i <= 5 ; i++) {
    if (i == 3) continue;
    System.out.println(i);
}
```

Output :
1 2 4 5

8. ARRAYS IN JAVA

"Array is a collection of similar data elements stored in contiguous memory locations."

1. WHAT IS AN ARRAY ?

→ Array is a data structure that stores multiple values of the same type.
→ Elements in array are stored in contiguous memory locations.
→ Each element is accessed using an index.
→ Index in Java always starts from 0.

2. WHY ARRAYS ARE USED ?

- Store multiple values under a single name.
- Easy to access using index.
- Improves code efficiency.
- Forms the basis for other data structures.
- Used extensively in real world applications.

3. DECLARATION & INITIALIZATION

Type	Syntax	Example
Declaration	dataType[] arrayName;	int[] arr;
Initialization	arrayName = new dataType[size];	arr = new int[5];
Declaration + Initialization	dataType[] arrayName = new dataType[size];	int[] arr = new int[5];

4. 1-D ARRAY (ONE DIMENSIONAL)

Example : int[] arr = new int[5];

Index :	0	1	2	3	4
arr :	10	20	30	40	50

- Size of array is fixed.
- All elements are of same data type.
- Accessed using arr[index].

5. TRAVERSING 1-D ARRAY

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

6. 2-D ARRAY (TWO DIMENSIONAL)

Example : int[][] mat = new int[3][4];

Index :	0	1	2	3
0				
1				
2				

Accesss :
mat[row][column]
row index, column index

7. TRAVERSING 2-D ARRAY

```
for (int i = 0; i < mat.length; i++) {
    for (int j = 0; j < mat[i].length; j++) {
        System.out.print(mat[i][j] + " ");
    }
    System.out.println();
}
```

8. ARRAY LENGTH

- Length of array is fixed and can be found using length property.

```
int[] arr = new int[5];
int size = arr.length;  // 5
```

9. ENHANCED FOR LOOP (for-each)

```
for (int num : arr) {
    System.out.println(num);
}
```

- Simplifies traversal.
- Works only for arrays (not for 2-D easily).

10. COMMON ARRAY ERRORS

- ArrayIndexOutOfBoundsException
  → Accessing invalid index.
- NegativeArraySizeException
  → Negative size.
- NullPointerException
  → Using array before initialization.

11. BASIC ARRAY OPERATIONS

Operation	Description	Example
1. Access	Access element using index	arr[2]
2. Update	Update element at an index	arr[2] = 100;
3. Traversal	Visit each element	Using for loop / for-each loop
4. Search	Search an element	Linear search
5. Insert	Insert element (shift elements)	Insert at position
6. Delete	Delete element (shift elements)	Remove element
7. Sort	Arrange elements in order	Ascending / Descending

12. QUICK NOTES

★ Array size is fixed once created.
★ All elements are stored in continuous memory locations.
★ Arrays are objects in Java.
★ Multi-dimensional arrays are arrays of arrays.
★ Useful in solving real world problems like matrices, tables, lists, etc.

Example 1 (Max in Array) : Input → [3, 7, 1, 9, 4] Output → 9 (max element)
Example 2 (Sum in Array) : Input → [2, 4, 6, 8] Output → 20 (sum of elements)

9. METHODS IN JAVA

"Method is a block of code that performs a specific task and can be reused."

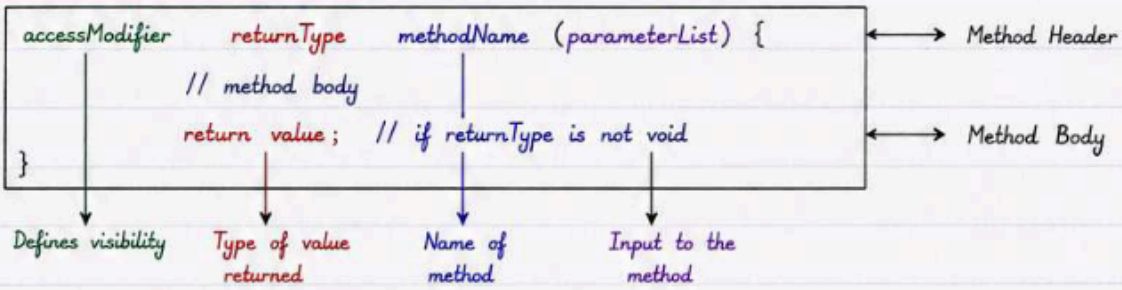
1. WHAT IS A METHOD ?

→ A method is a collection of statements that perform a specific task.
→ It helps in code reusability and modularity.
→ Every Java program must have at least one method : main().

2. ADVANTAGES OF METHODS

- Promotes code reusability.
- Improves readability and maintainability.
- Reduces code duplication.
- Breaks large program into smaller modules.
- Makes testing and debugging easier.

3. ANATOMY OF A METHOD



4. TYPES OF METHODS

Type	Description	Example
1. Built-in Methods	Provided by Java library	System.out.println()
2. User-defined Methods	Created by programmer	myMethod()
3. Static Methods	Belong to class, called without object	Math.sqrt()
4. Instance Methods	Belong to object, called using object	obj.display()

5. METHOD WITH PARAMETERS

```
int add(int a, int b) {        ← Parameters
    int sum = a + b;
    return sum;
}
```

Example :

```
int result = add(10, 20); // arguments
```

6. METHOD WITHOUT PARAMETERS

```
void greet() {
    System.out.println("Hello Aman!");
}
```

Example :

```
greet(); // no arguments
```

7. METHOD WITH RETURN VALUE

```
int multiply(int a, int b) {
    int prod = a * b;
    return prod;
}
```

Example :

```
int p = multiply(5, 6); // returns 30
```

8. VOID vs NON-VOID METHODS

Feature	Void Method	Non-Void Method
Return Type	Returns nothing (void)	Returns a value (int, String, etc.)
Use	Performs an action	Performs an action and returns result
'return' keyword	Not required	Required to return value
Example	void display() {}	int add() { return a + b; }

QUICK NOTES

★ Method name should be meaningful and follow naming conventions.
★ Methods can call other methods.
★ Java is call-by-value.
★ 'return' ends the method execution.

9. METHOD OVERLOADING (SAME NAME, DIFFERENT PARAMETERS)

```
int add(int a, int b) {
    return a + b;
}
int add(int a, int b, int c) {
    return a + b + c;
}
```

Rules :

- Same method name.
- Different parameter list (type / number / order).
- Return type can be same or different.

Example :

```
int x = add(10, 20);      // calls first add()
int y = add(10, 20, 30); // calls second add()
System.out.println(x);    // 30
System.out.println(y);    // 60
```

Summary : Methods make code modular, reusable and easy to maintain.

10. STRINGS IN JAVA

"String is a sequence of characters used to represent text."

1. WHAT IS STRING ?

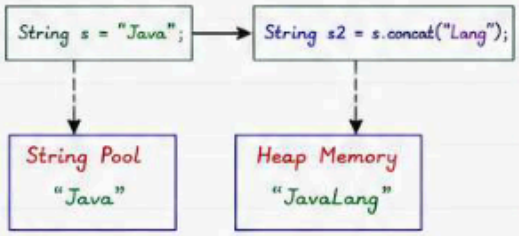
→ String is a class in java.lang package.
→ It is used to handle text.
→ Strings are immutable (cannot be changed).
→ String literal is stored in String Pool.

2. CREATING STRINGS

Way	Example	Note
1. String Literal	`String s1 = "Hello";`	Stored in String Pool
2. new Keyword	`String s2 = new String("Hello");`	Stored in Heap
3. char Array	`char ch[] = {'H','i'};` `String s3 = new String(ch);`	From char array

3. STRING IMMUTABILITY

→ Once a String object is created, it cannot be modified.
→ Any change creates a new String object.



Original String "Java" remains unchanged.

4. COMMON STRING OPERATIONS

Method	Description	Example
length()	Returns length of string	s.length() → 5
charAt(i)	Returns character at index i	s.charAt(0) → 'H'
substring(i, j)	Returns substring from i to j-1	s.substring(1, 4) → "ell"
concat(str)	Joins two strings	s.concat("World")
equals(str)	Compares content	s.equals("Test")
equalsIgnoreCase(str)	Compares without case	s.equalsIgnoreCase()
toUpperCase()	Converts to upper case	s.toUpperCase()
toLowerCase()	Converts to lower case	s.toLowerCase()
trim()	Removes leading/trailing spaces	s.trim()
replace(old, new)	Replaces old with new	s.replace('a','@')

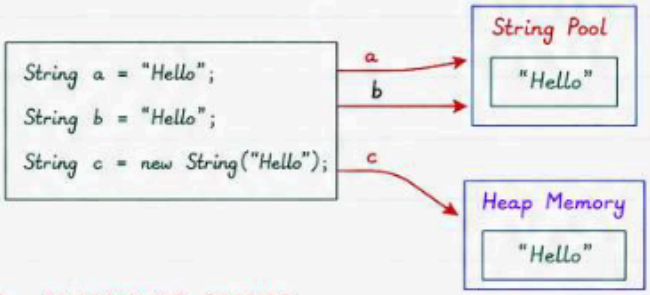
5. STRING COMPARISON

→ Use equals() to compare content.
→ Use == to compare references (not content).

Expression	Result	Reason
"Java".equals("Java")	true	Same content
"Java" == "Java"	true	Same literal in pool
new String("Java") == new String("Java")	false	Different objects in heap
s1.equals(s2)	true	Same content

6. STRING POOL

→ String literals are stored in String Pool.
→ If same literal is created again, same reference is used.



7. STRING CONCATENATION

→ Using + operator.
→ Using concat() method.

```
String s1 = "Hello";
String s2 = "World";
String s3 = s1 + " " + s2;
String s4 = s1.concat(" ").concat(s2);
System.out.println(s3);   // Hello World
System.out.println(s4);   // Hello World
```

8. IMPORTANT POINTS

★ Strings are immutable.
★ String literals are stored in String Pool.
★ Use equals() for content comparison.
★ Use == for reference comparison.
★ String is widely used in real world applications. (e.g., form data, messages, file handling etc.)

QUICK EXAMPLES

1. Length of string

```
String s = "JavaLang";
System.out.println(s.length());
// Output: 8
```

2. Substring

```
String s = "Hello World";
System.out.println(s.substring(6));
// Output: World
```

3. Replace

```
String s = "banana";
System.out.println(s.replace('a','@'));
// Output: b@n@n@
```

Summary : Strings are immutable objects used to store and manipulate text in Java.

11. STRINGBUILDER & STRINGBUFFER

"StringBuilder and StringBuffer are mutable classes used to modify strings efficiently."

1. WHAT ARE THEY ?

→ Both are used to create and modify mutable (changeable) strings.
→ They are more efficient than String class for many modifications.
→ StringBuilder is non-synchronized (faster).
→ StringBuffer is synchronized (thread-safe).

2. STRINGBUILDER vs STRINGBUFFER

Feature	StringBuilder	StringBuffer
Synchronization	Not synchronized (Not thread-safe)	Synchronized (Thread-safe)
Performance	Faster	Slower
Use	Single-threaded environment	Multi-threaded environment
Since	Java 1.5	Java 1.0

3. COMMON METHODS (APPLICABLE TO BOTH)

Method	Description
append()	Adds the given value at the end.
insert(int index, value)	Inserts the value at the specified index.
delete(int start, int end)	Deletes characters from start (inclusive) to end (exclusive).
replace(int start, int end, value)	Replaces characters from start to end with the given value.
reverse()	Reverses the sequence.
toString()	Converts to normal String.

4. EXAMPLE

```
StringBuilder sb = new StringBuilder("Aman");
sb.append(" Views");                    // Aman Views
sb.insert(0, "Hello ");                 // Hello Aman Views
sb.delete(6, 11);                       // Hello Views
sb.replace(6, 11, "World");             // Hello World
sb.reverse();                           // dlroW olleH
String result = sb.toString();          // dlroW olleH
```

12. STRING FORMATTING & USEFUL METHODS

1. STRING FORMATTING

→ Java provides different ways to format strings.

Method	Syntax	Example
1. Concatenation	str1 + str2 + ...	"Hello" + " " + "Aman"
2. String.format()	String.format("format", args...)	String.format("Hi %s, age %d", "Aman", 25)
3. printf()	System.out.printf("format", args...)	System.out.printf("Value = %d", 100)

2. COMMONLY USED METHODS

Method	Description
charAt(index)	Returns character at the given index.
indexOf(str)	Returns index of first occurrence of the substring.
lastIndexOf(str)	Returns index of last occurrence of the substring.
substring(start, end)	Returns part of the string (start inclusive, end exclusive).
toUpperCase()	Converts string to upper case.
toLowerCase()	Converts string to lower case.
trim()	Removes leading and trailing spaces.
replace(old, new)	Replaces all occurrences of old with new.
split(regex)	Splits string based on the given regex.
contains(str)	Checks if string contains the given substring.
startsWith(str)	Checks if string starts with the given substring.
endsWith(str)	Checks if string ends with the given substring.

3. EXAMPLE

```
String text = "  Hello Aman Views  ";          // original string (with spaces)
System.out.println(text.trim());               // Hello Aman Views
System.out.println(text.toUpperCase());        // HELLO AMAN VIEWS
System.out.println(text.toLowerCase());        // hello aman views
System.out.println(text.charAt(6));            // A
System.out.println(text.indexOf("Aman"));      // 8
System.out.println(text.replace("Aman", "Java")); //  Hello Java Views
String[] parts = text.trim().split(" ");       // split by space
System.out.println(parts.length);              // 3
```

★ Strings in Java are powerful and immutable. Proper use of methods makes code clean and efficient.

13. EXCEPTION HANDLING IN JAVA

"Exception handling allows us to handle runtime errors and maintain normal flow of the program."

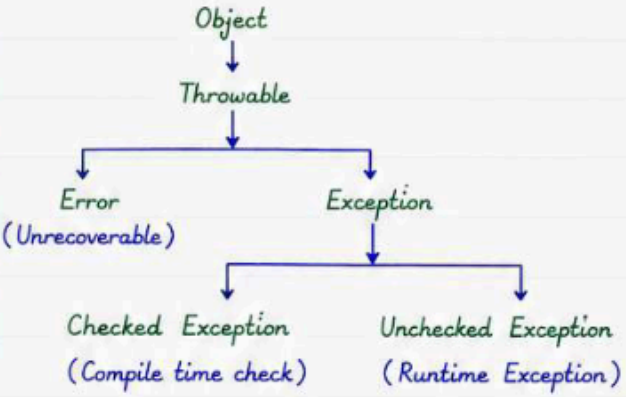
1. WHAT IS EXCEPTION ?

→ Exception is an unwanted or unexpected event that disrupts the normal flow of the program.

→ It occurs during runtime.

→ Java provides a mechanism to handle exceptions gracefully using try-catch.

2. EXCEPTION HIERARCHY



3. CHECKED vs UNCHECKED EXCEPTIONS

Feature	Checked Exception	Unchecked Exception
Checked at	Compile time	Runtime
Handling	Must be handled	May or may not be handled
Superclass	Exception	RuntimeException
Examples	IOException, SQLException, ClassNotFoundException	NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException

4. TRY–CATCH–FINALLY

```
try {
    // Code that may throw exception
} catch (Exception e) {
    // Handle exception
} finally {
    // Code that always executes
}
```

← try → Contains the code which might throw exception.

← catch → Handles the exception.

← finally → Executes always (optional). Used for cleanup code.

5. EXAMPLE

```
int a = 10, b = 0, result;
try {
    result = a / b;   // may throw exception
    System.out.println("Result: " + result);
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("Finally block executed");
}
```

OUTPUT :

```
Cannot divide by zero
Finally block executed
```

14. ARRAYS & VARARGS IN JAVA

"Arrays store multiple values of the same type.
Varargs allows method to accept variable number of arguments."

1. ARRAYS IN JAVA

→ Array is a container object that holds fixed number of values of same type.

→ Index starts from 0.

→ Length is fixed (cannot grow or shrink).

→ Stored in contiguous memory location.

2. DECLARING & INITIALIZING ARRAY

```
dataType[] arrayName = new dataType[size];
                or
dataType[] arrayName = {val1, val2, val3, ...};
```

3. ARRAY EXAMPLE

```
int[] marks = {85, 90, 78, 92, 88};
System.out.println(marks[0]);   // 85
System.out.println(marks[3]);   // 92
```

4. COMMON ARRAY OPERATIONS

Operation	Syntax	Example	Note
Length	arr.length	int n = arr.length;	Returns size of array
Access	arr[index]	int x = arr[2];	Gets value at index
Update	arr[index] = value	arr[1] = 100;	Changes value at index
Traverse	for loop	for(int i=0;i<arr.length;i++) System.out.println(arr[i]);	Iterate all elements
Multi-dimensional	dataType[][] arr	int[][] a = new int[3][4];	Array of arrays

5. FOR-EACH LOOP (ENHANCED FOR)

```
int[] arr = {10, 20, 30, 40};
for (int num : arr) {
    System.out.println(num);
}                          // prints all elements
```

6. VARARGS (VARIABLE-ARGUMENTS)

→ Allows a method to accept zero or more arguments.

→ Uses "..." after data type.

→ Must be the last parameter in method.

```
returnType methodName(dataType... args) {
        // method body
}
```

7. VARARGS EXAMPLE

```
void sum(int... numbers) {
    int total = 0;
    for(int n : numbers) {
        total += n;
    }
    System.out.println("Sum = " + total);
}
```

```
sum();                  // Sum = 0
sum(10);                // Sum = 10
sum(10, 20, 30);        // Sum = 60
sum(5, 15, 25, 35);     // Sum = 80
```

8. VARARGS vs ARRAY

Feature	Array	Varargs
Purpose	Store multiple values	Pass multiple arguments to method
Length	Fixed size	Can be zero or more
Usage	Used as data structure	Used in method parameters
Syntax	int[] arr	int... arr
Example	int[] a = {1,2,3};	void m(int... a)

Summary : Arrays store fixed size data. Varargs makes methods flexible by accepting variable number of arguments.

15.

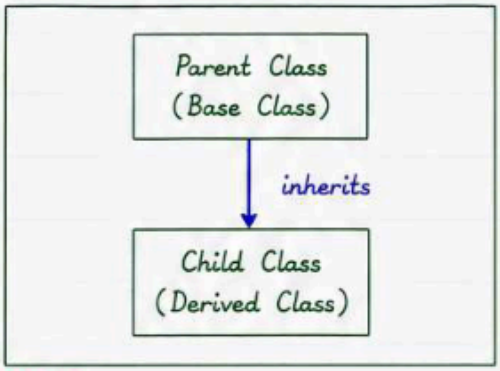
OOPs CONCEPT - 1

INHERITANCE

Definition : Inheritance is a mechanism in which one class acquires the properties and behavior (methods) of another class. It promotes code reusability.

1. WHY INHERITANCE ?

- → Code reusability
- → Reduces duplication
- → Improves maintainability
- → Creates a natural relationship between classes



2. SYNTAX

```
class Parent {
    // fields
    // methods
}
class Child extends Parent {
    // own fields
    // own methods
}
```

3. EXAMPLE

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();      // Inherited from Animal
        d.bark();       // Own method of Dog
    }
}
```

4. KEY POINTS

- → Child class (Derived class) extends Parent class (Base class) using the keyword **extends**.
- → Child class inherits all non-private members of Parent class.
- → Child class can have its own fields and methods.
- → Inheritance supports the "is-a" relationship.

5. TYPES OF INHERITANCE (in Java)

Single	Multilevel	Hierarchical	Multiple	Hybrid
One child class inherits from one parent class.	A child class becomes parent for another class.	Multiple child classes inherit from the same parent class.	A class inherits from multiple parent classes.	Combination of two or more types.
A → B	A → B → C	A → B , A → C	(Not supported in Java)	(Achieved using interfaces)

★ **Note :** In Java, multiple inheritance is achieved using Interfaces, not with classes.

16.

OOPs CONCEPT - 2

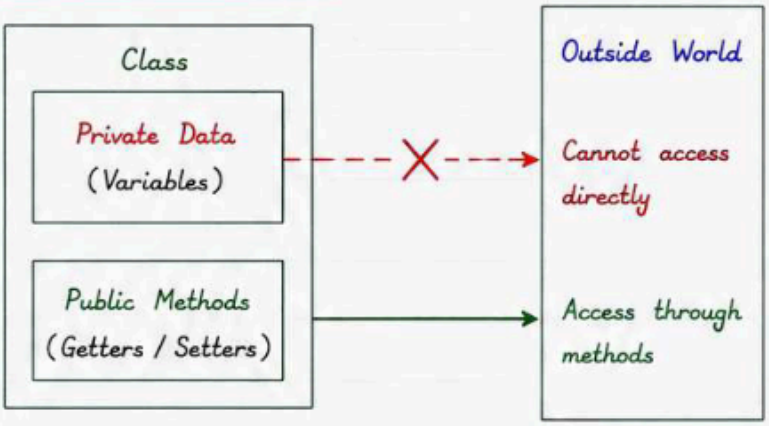
ENCAPSULATION

Definition : Encapsulation is the mechanism of wrapping data (variables) and code (methods) together as a single unit and restricting direct access to some of the object's components.

1. WHY ENCAPSULATION ?

- → Protects the data from outside access.
- → Improves code security.
- → Provides better control over data.
- → Helps in hiding internal implementation.

2. HOW IT WORKS ?



3. SYNTAX (in Java)

```
class Student {
    private String name;   // private data
    private int age;       // private data

    // public getter method
    public String getName() {
        return name;
    }

    // public setter method
    public void setName(String n) {
        name = n;
    }
}
```

4. EXAMPLE

```
public class BankAccount {
    private double balance;

    public double getBalance() {
        return balance;
    }

    public void setBalance(double amount) {
        if (amount >= 0) {
            balance = amount;
        }
    }
}
// Usage
BankAccount acc = new BankAccount();
acc.setBalance(5000);
System.out.println(acc.getBalance());   // 5000
```

5. KEY POINTS

- ★ Use "private" to hide data.
- ★ Provide "public" methods to access and modify data.
- ★ Getters are used to read data.
- ★ Setters are used to update data with validation if needed.
- ★ Encapsulation makes the class flexible and easy to maintain.

Remember :
Hide the data, expose the behavior.

17.

OOPs CONCEPT - 3

POLYMORPHISM

Definition : Polymorphism means "many forms". It allows objects of different classes to be treated as objects of a common base class.

1. WHY POLYMORPHISM ?

- → Allows one interface for different types.
- → Improves code flexibility.
- → Easier to extend and maintain.
- → Built on inheritance.

2. TYPES OF POLYMORPHISM

1. Compile Time (Static) Polymorphism
   Method Overloading
2. Runtime (Dynamic) Polymorphism
   Method Overriding

3. COMPILE TIME POLYMORPHISM (METHOD OVERLOADING)

Same method name, different parameters (within the same class).

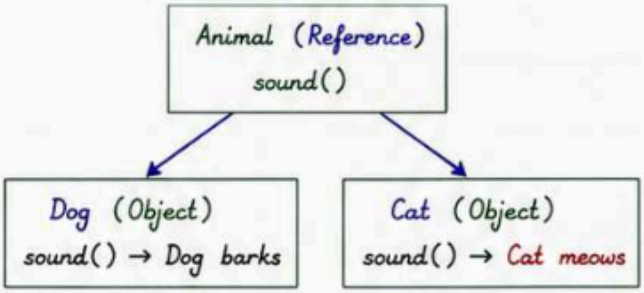
```
class Math {
    int add(int a, int b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

4. RUNTIME POLYMORPHISM (METHOD OVERRIDING)

Same method name and parameters, but different implementation (in child class).

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

5. POLYMORPHISM IN ACTION (RUNTIME)




```
Animal a = new Dog();   // Upcasting
a.sound();              // Dog barks

Animal b = new Cat();   // Upcasting
b.sound();              // Cat meows
```

★ **Summary :** Polymorphism lets us use a common interface and get different behaviors depending on the object.

18.

OOPs CONCEPT - 4

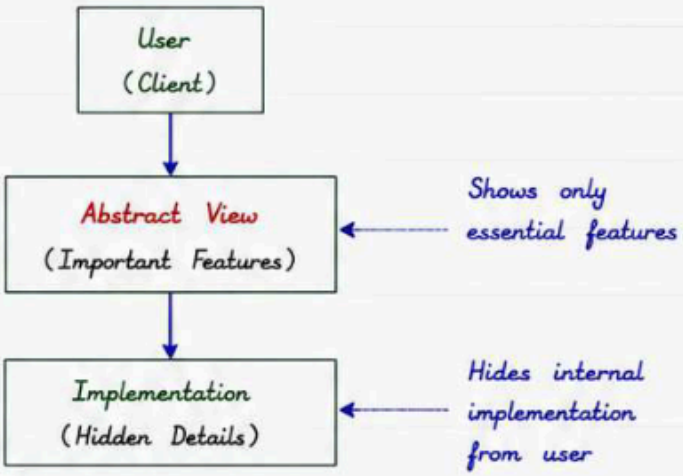
ABSTRACTION

Definition : Abstraction is the process of hiding implementation details and showing only essential features to the user. It reduces complexity and improves security.

1. WHY ABSTRACTION ?

- → Hides unnecessary details.
- → Reduces complexity.
- → Focus on what an object does, not how it does.
- → Improves security and maintainability.

2. HOW ABSTRACTION WORKS ?



3. SYNTAX (in Java)

```
abstract class Shape {
    abstract void draw();  // abstract method
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing Circle");
    }
}
```

4. EXAMPLE

```
abstract class Shape {
    abstract void draw();
}
class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing Rectangle");
    }
}
public class Main {
    public static void main(String[] args) {
        Shape obj = new Rectangle();   // Abstraction
        obj.draw();                    // Output: Drawing Rectangle
    }
}
```

5. KEY POINTS

- ★ Achieved using abstract classes and interfaces.
- ★ Abstract class can have abstract and non-abstract methods.
- ★ Abstract method has no body.
- ★ Child class must provide implementation.
- ★ Helps in reducing complexity and increasing security.

6. ABSTRACTION vs ENCAPSULATION

Feature	Abstraction	Encapsulation
Focus	Hiding implementation details	Hiding data and controlling access
Keywords	abstract, interface	private, getter, setter
Purpose	Shows what to do	Hides how it is done
Level	Design level	Implementation level

★ **Summary :** Abstraction helps in exposing only important features and hiding the internal details from the user.